

EX.NO:1	STUDY AND INSTALLATION OF FLUTTER MULTI-PLATFORM ENVIRONMENT
DATE:	

AIM:

To study and installation of flutter Multi-Platform Environment.

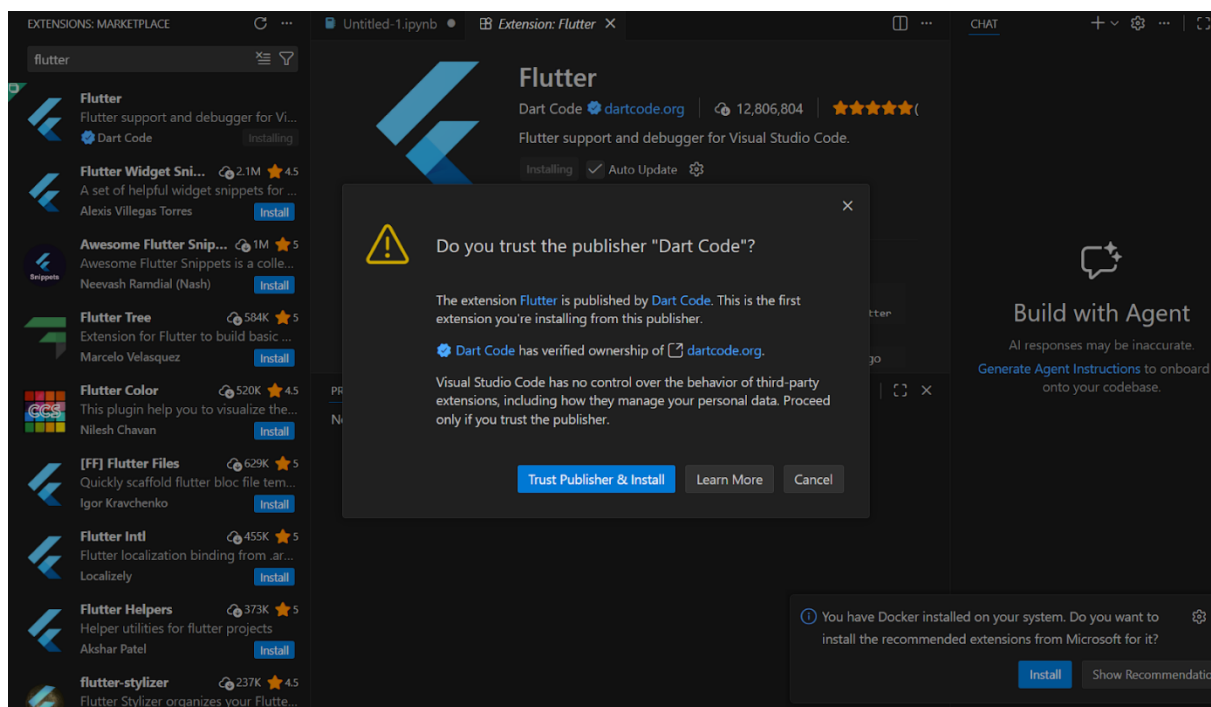
PROCEDURE:

Step1: Install Microsoft VS Code

Step2: Click on View->Extensions (Ctrl + Shift+X)

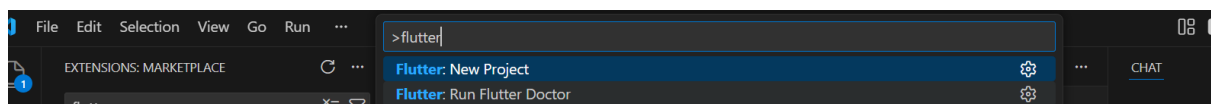
Search for Flutter

Click Install



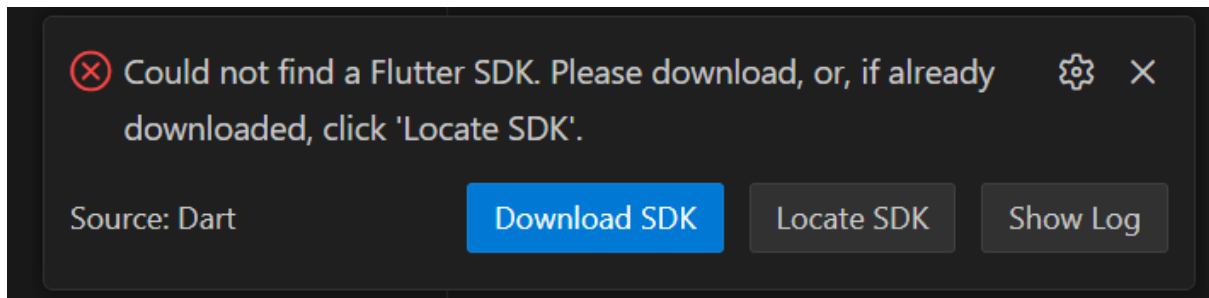
Step3: Click on View->Command Palette(Ctrl+shift+P)

Type Flutter and Select New Project



Step4: Install Flutter SDK as prompted by VSCode.

Click on Download SDK.



After Installation, restart VSCode.

Click on View->Command Palette(Ctrl+Shift+P)

Type Flutter and Select New Project

Select Empty Application and choose the project folder

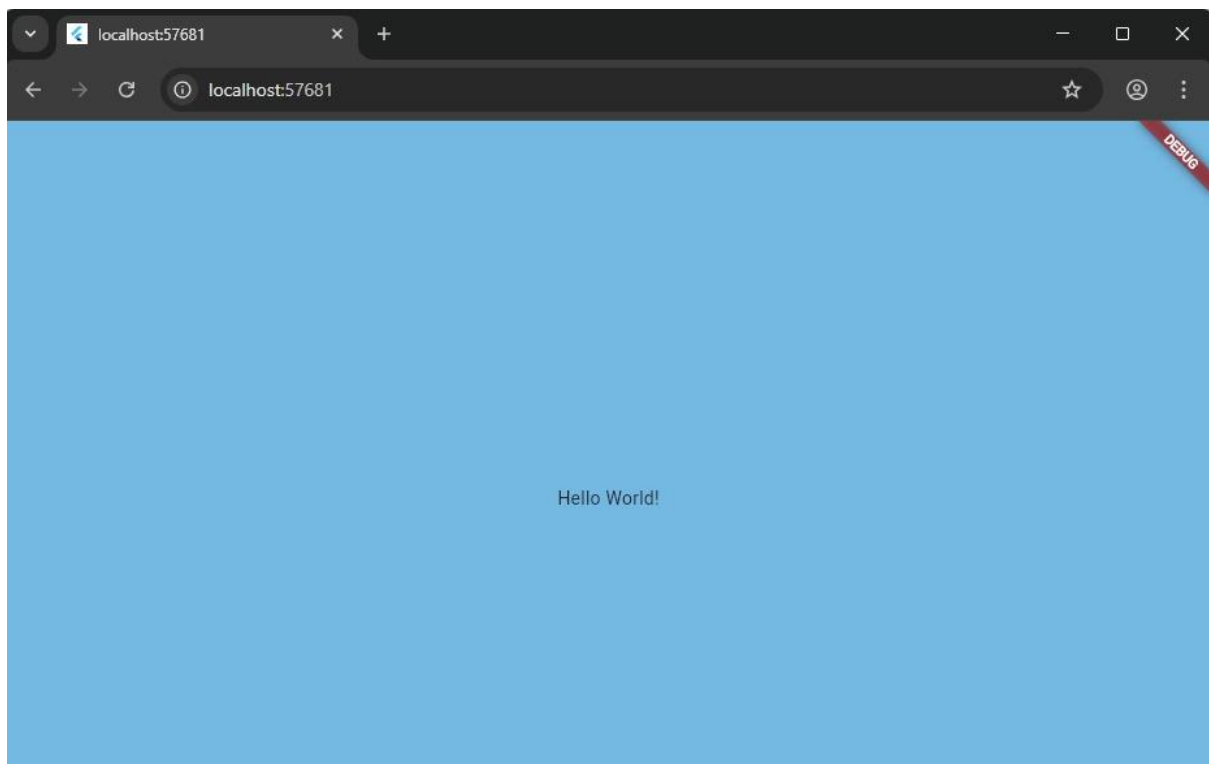
Step 5: From the created project folder

Open lib->main.dart

Click on Run tab->Run without Debugging

Select device to run.

Select Chrome



RESULT:

Thus, the study and installation of Flutter in Multi-Platform Environment was written and executed successfully.

EX.NO:2	DEVELOP AN APPLICATION THAT USES WIDGETS,GUI COMPONENTS, FONTS AND COLORS
DATE:	

AIM:

To design and develop an application using widgets and GUI components by applying appropriate fonts and color combinations to create an interactive, user-friendly, and visually appealing interface.

PROCEDURE:

1. Change the Background of the Application:

The background color of the application is set using the `Scaffold` widget to give a soft pink theme suitable for a stationery store interface.

dart

```
Scaffold(
  backgroundColor: Colors.pink[50],
);
```

2.Appropriate Title Related to the Project

The application title “PenzY” is displayed at the top using the `AppBar` widget with suitable styling.

dart

```
AppBar(
  title: Text('PenzY'),
  backgroundColor: Colors.pinkAccent,
);
```

3.Images Related to the Project

Relevant images for sections like New Arrivals, For You, and Offers are added using the `Image.asset` widget.

dart

```
Image.asset(  
  'assets/image.png',  
  height: 120,  
);
```

4. About the App / Organization

A short description is provided under each section explaining the purpose, such as showcasing stationery collections and offers.

dart

```
Text(  
  'Discover our newest collection of stationery items...',  
);
```

5. Headings for Each Section

Section headings like “New Arrivals”, “For You”, and “Offers” are displayed using bold text to clearly separate content.

dart

```
Text(  
  'New Arrivals',  
  style: TextStyle(fontWeight: FontWeight.bold),  
);
```

6. Different Color for Heading and Text

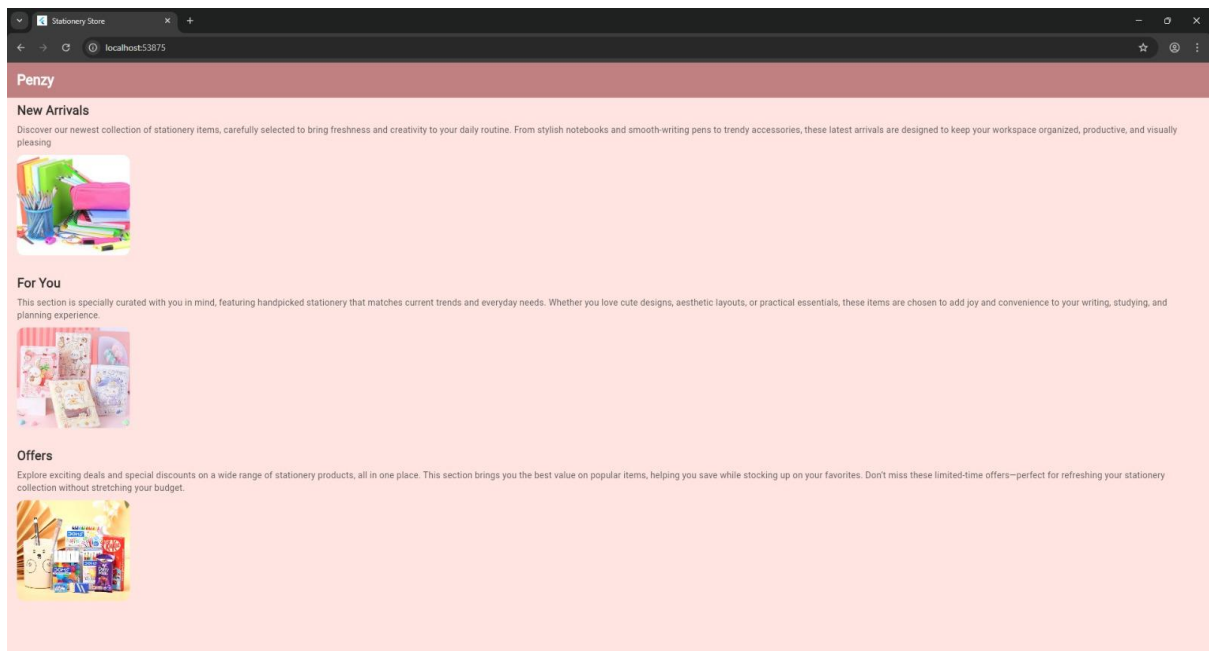
Different colors are applied to headings and normal text to improve readability and visual appearance.

dart

```
Text(  
  'Heading',  
  style: TextStyle(color: Colors.black),  
);
```

```
Text(  
    'Content text',  
    style: TextStyle(color: Colors.grey),  
);
```

OUTPUT:



RESULT:

The application was successfully developed using widgets and GUI components. It displays sections like New Arrivals, For You, and Offers with images and descriptions. The interface is visually appealing with proper use of fonts and colors.

EX.NO:3	DEVELOP AN APPLICATION USING TEXT FIELDS,BUTTON AND NAVIGATION
DATE:	

AIM:

To design and implement a home page with Login and Sign-up buttons, and to develop separate pages for user authentication, where the Login page accepts username and password, and the Sign-up page collects user details such as username, password, date of birth, email, and phone number.

SAMPLE CODE FOR PAGE NAVIGATION

```
class AuthScreen extends StatefulWidget {  
  
  final void Function(String username) onLoginSuccess;  
  
  const AuthScreen({Key? key, required this.onLoginSuccess}) : super(key: key);  
  
  @override  
  State<AuthScreen> createState() => _AuthScreenState();  
}
```



```
class _AuthScreenState extends State<AuthScreen>  
  with SingleTickerProviderStateMixin {  
  
  bool _isLogin = true;  
  
  
  final _userCtrl = TextEditingController();  
  final _passCtrl = TextEditingController();  
  final _confirmCtrl = TextEditingController();  
  
  
  bool _obscurePass = true;  
  bool _obscureConfirm = true;  
  bool _loading = false;  
  String? _errorMsg;
```

```

late AnimationController _animCtrl;

late Animation<double> _fadeAnim;

late Animation<Offset> _slideAnim;

@override
void initState() {
  super.initState();

  _animCtrl = AnimationController(
    vsync: this, duration: const Duration(milliseconds: 500));

  _fadeAnim = CurvedAnimation(parent: _animCtrl, curve: Curves.easeOut);

  _slideAnim = Tween<Offset>(begin: const Offset(0, 0.08), end: Offset.zero)
    .animate(CurvedAnimation(parent: _animCtrl, curve: Curves.easeOut));

  _animCtrl.forward();
}

@override
void dispose() {
  _animCtrl.dispose();
  _userCtrl.dispose();
  _passCtrl.dispose();
  _confirmCtrl.dispose();
  super.dispose();
}

void _switchMode() {
  setState() {
    _isLogin = !_isLogin;
    _errorMsg = null;
    _passCtrl.clear();
  }
}

```

```
        _confirmCtrl.clear();
    });
    _animCtrl.forward(from: 0);
}

void _submit() async {
    final username = _userCtrl.text.trim();
    final password = _passCtrl.text.trim();

    if (username.isEmpty || password.isEmpty) {
        setState(() => _errorMsg = 'Please fill in all fields.');
```

return;

```
    }

    if (username.length < 3) {
        setState(() => _errorMsg = 'Username must be at least 3 characters.');
```

return;

```
    }

    if (password.length < 6) {
        setState(() => _errorMsg = 'Password must be at least 6 characters.');
```

return;

```
    }

    if (!_isLogin) {
        if (_confirmCtrl.text.trim() != password) {
            setState(() => _errorMsg = 'Passwords do not match.');
```

return;

```
        }
    }
}

setState() {
```



```
    _loading = true;
    _errorMsg = null;
  });

  await Future.delayed(const Duration(milliseconds: 600));

  if (_isLogin) {
    if (UserStore.login(username, password)) {
      widget.onLoginSuccess(username);
    } else {
      setState(() {
        _loading = false;
        _errorMsg = UserStore.exists(username)
          ? 'Incorrect password. Please try again.'
          : 'No account found with that username.';
      });
    }
  } else {
    if (UserStore.register(username, password)) {
      widget.onLoginSuccess(username);
    } else {
      setState(() {
        _loading = false;
        _errorMsg = 'Username already taken. Please choose another.';
      });
    }
  }
}
```

@override

Widget build(BuildContext context) {

 return Scaffold(

 backgroundColor: const Color(0xFFFF5F6FA),

 body: SafeArea(

 child: Center(

 child: SingleChildScrollView(

 padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),

 child: FadeTransition(

 opacity: _fadeAnim,

 child: SlideTransition(

 position: _slideAnim,

 child: Column(

 mainAxisSize: MainAxisSize.min,

 children: [

 Container(

 width: double.infinity,

 padding: const EdgeInsets.symmetric(

 vertical: 28, horizontal: 20),

 decoration: BoxDecoration(

 color: const Color(0xFF1A1A2E),

 borderRadius: BorderRadius.circular(24),

),

 child: Column(children: const [

 Icon(Icons.menu_book_rounded,

 size: 36, color: Color(0xFFFF6B554)),

 SizedBox(height: 12),

 Text('Stationery Shop',

 style: TextStyle(

```
        color: Colors.white,
        fontSize: 22,
        fontWeight: FontWeight.bold)),
    ),
    const SizedBox(height: 16),
    Container(
      width: double.infinity,
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(20),
      ),
      child: Column(
        children: [
          Row(children: [
            Expanded(
              child: TextButton(
                onPressed: _switchMode,
                child: const Text('Login')),
            Expanded(
              child: TextButton(
                onPressed: _switchMode,
                child: const Text('Sign Up')),
            ),
          ],
        ),
        TextField(
```

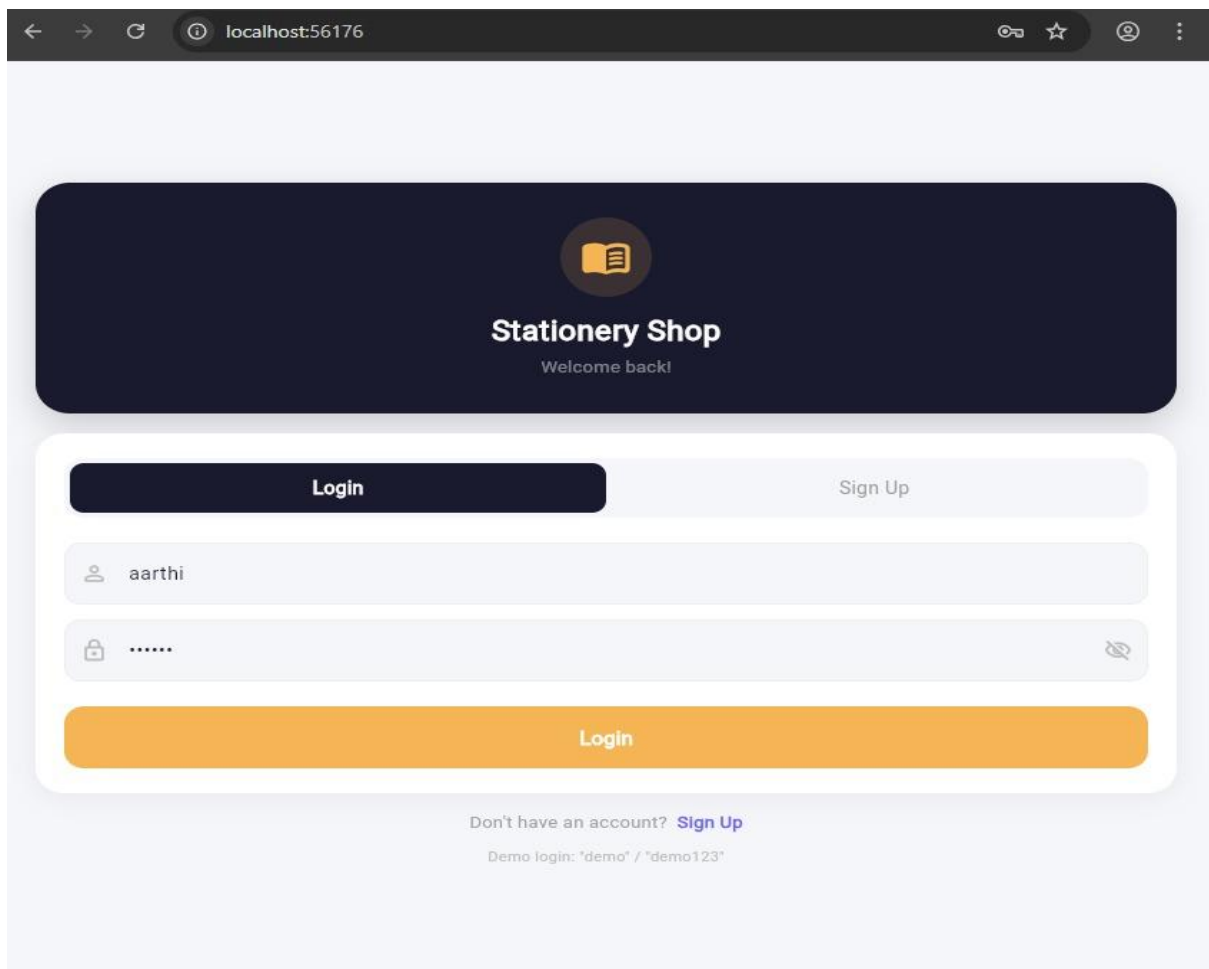
```
        controller: _userCtrl,  
        decoration:  
          const InputDecoration(hintText: 'Username'),  
      ),  
  
      TextField(  
        controller: _passCtrl,  
        obscureText: true,  
        decoration:  
          const InputDecoration(hintText: 'Password'),  
      ),  
  
      if (!_isLogin)  
        TextField(  
          controller: _confirmCtrl,  
          obscureText: true,  
          decoration: const InputDecoration(  
            hintText: 'Confirm Password'),  
          ),  
  
      const SizedBox(height: 20),  
  
      ElevatedButton(  
        onPressed: _submit,  
        child: Text(_isLogin ? 'Login' : 'Sign Up'),  
      ),  
    ],  
  ),  
),
```

```

        ],
      ),
    ),
  ),
),
),
),
);
}
}

```

OUTPUT:



The screenshot displays a web browser window with the address bar showing 'localhost:56176'. The page features a dark blue header with a book icon and the text 'Stationery Shop' and 'Create your account'. Below the header, there are two tabs: 'Login' and 'Sign Up'. The 'Login' tab is active, showing a form with three input fields: a username field containing 'varshini', a password field with masked characters '.....', and a second password field also with masked characters '.....'. Each password field has an eye icon for toggling visibility. A large orange button labeled 'Create Account' is positioned below the password fields. At the bottom, there is a link for 'Already have an account? Login' and a note: 'Demo login: "demo" / "demo123"'. The browser's address bar and navigation icons are visible at the top.

RESULT:

The application was successfully developed with Login and Sign-up functionality. Users can enter credentials to log in or create a new account with validation. The interface provides a smooth and user-friendly authentication experience.

EX.NO:5	HANDLING GESTURES AND ACTIONS
DATE:	

AIM:

To design and develop a sample application that enables users to drag a color element and drop it into a designated area, demonstrating drag-and-drop functionality using GUI components.

STEPS TO CREATE DRAG AND DROP COMPONENT:

1. CREATE THE DRAGGABLE WIDGET

Define a widget that can be dragged and specify its properties.

a. **Data to be Passed**

Assign the data to be transferred during dragging (e.g., color and name).

data: {'color': color, 'name': name},

b. **Feedback**

Provide a widget that appears while dragging.

c. **Child**

Specify the content to be displayed in the draggable widget.

2. CREATE THE DRAG TARGET(DROP AREA)

Define the area where the draggable object can be dropped.

a. **DragTarget Object**

DragTarget<Map<String, dynamic>>(

b. **Accepting the Dragged Data**

Handle the data when it is dropped into the target area.

c. **Builder property - defines UI of drag target**

PROGRAM:

```
Positioned(
  left: 16,
  right: 16,
  bottom: 16,
  child: DragTarget<Map>(
    onWillAcceptWithDetails: (details) {
      setState(() => _isDragOver = true);
      return true;
    },
    onLeave: (_) => setState(() => _isDragOver = false),
    onAcceptWithDetails: (details) {
      setState(() => _isDragOver = false);
      addToCart(details.data);
    },
    builder: (context, candidateData, rejectedData) {
      final bool dragging = candidateData.isNotEmpty || _isDragOver;
      return GestureDetector(
        onTap: showCart,
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 200),
          curve: Curves.easeOut,
          height: dragging ? 90 : 72,
          decoration: BoxDecoration(
            color: dragging
              ? const Color(0xFFFF6B554)
              : const Color(0xFF1A1A2E),
            borderRadius: BorderRadius.circular(24),
          ),
        ),
```



```

        child: Row(
          children: [
            Icon(Icons.shopping_bag_outlined, color: Colors.white),
            Text("Cart"),
          ],
        ),
      ),
    );
  },
),
);

// Product Card
Widget _buildProductCard(Map product, String productId, double rating,
  int count, Map reviews) {
  return Container(
    margin: const EdgeInsets.all(16),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(20),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Image.asset("assets/images/${product["image"]}", height: 200),
        Text(product["name"],
          style: const TextStyle(fontWeight: FontWeight.bold)),
        Text("₹${product["price"]}"),

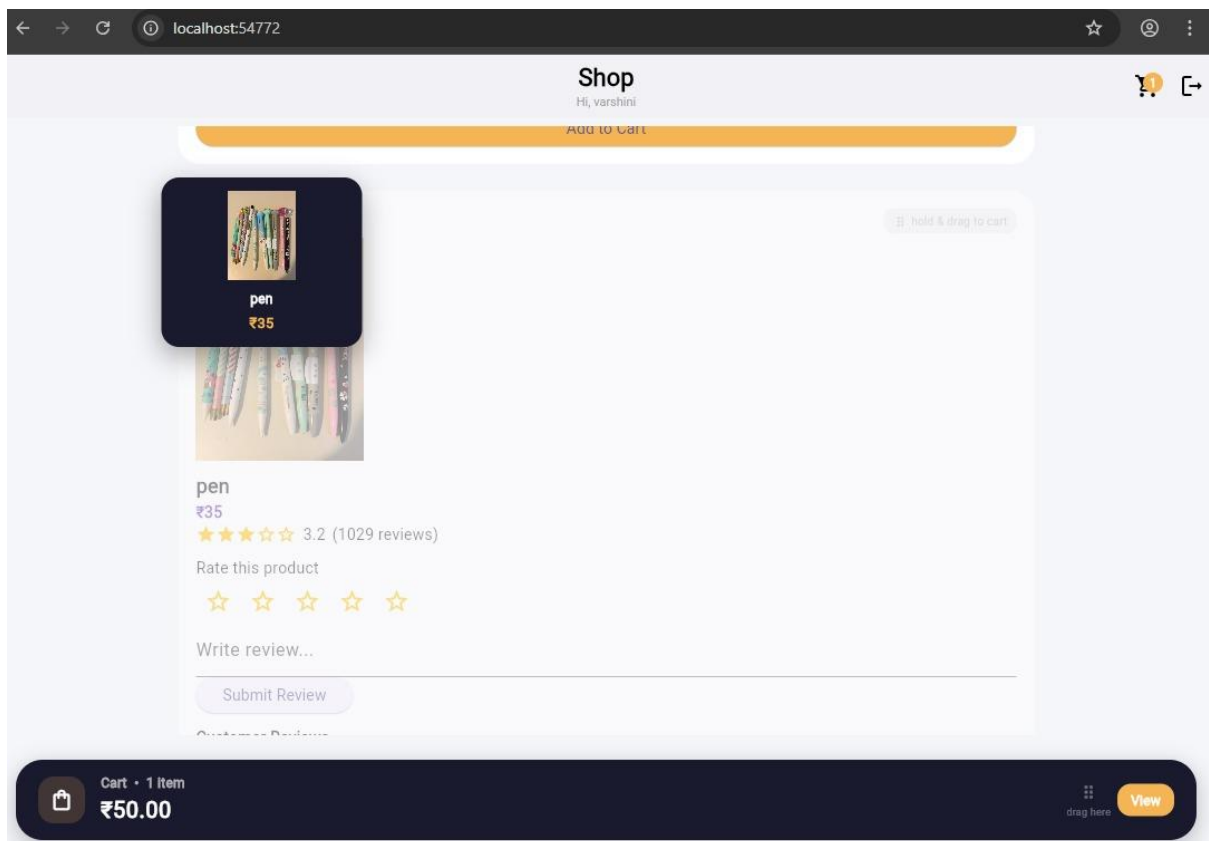
```

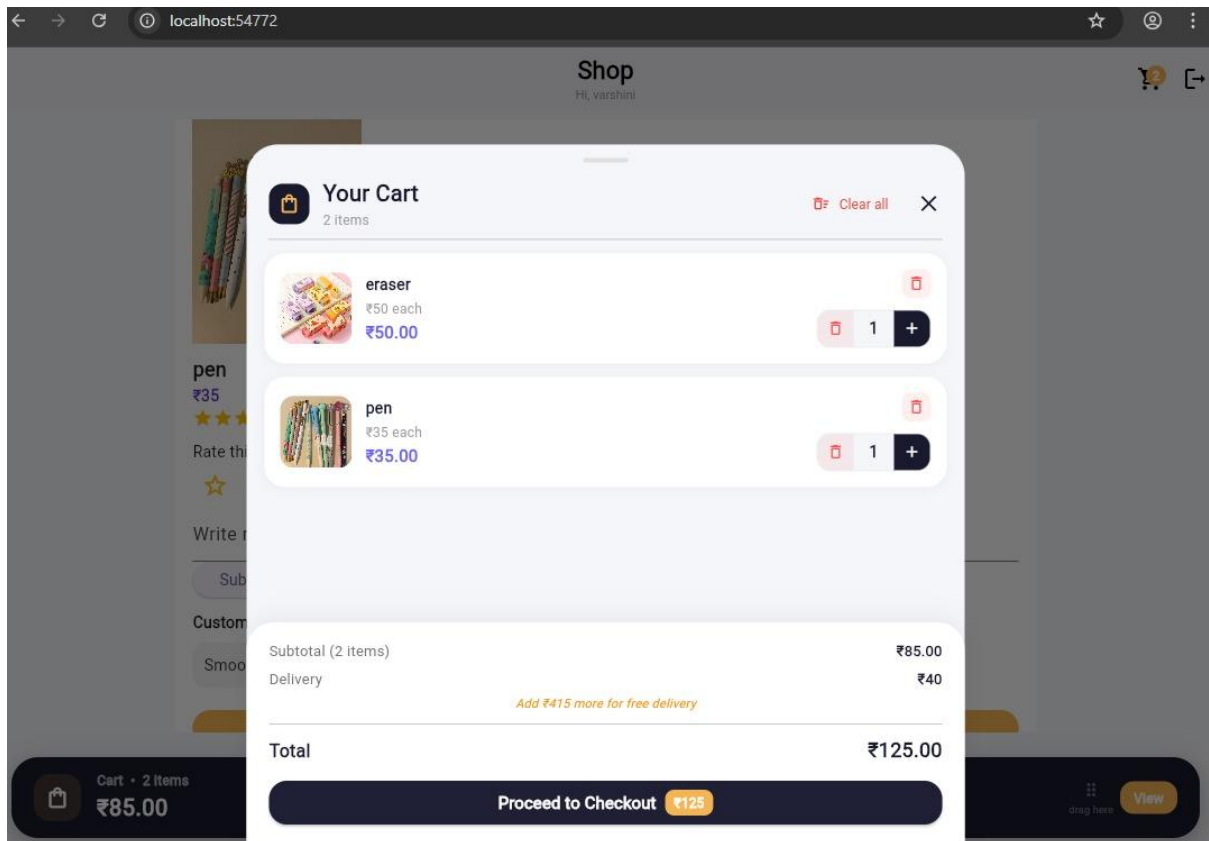
```

ElevatedButton(
  onPressed: () => addToCart(product),
  child: const Text("Add to Cart"),
),
],
),
);
}

```

OUTPUT:





RESULT:

The application was successfully developed with drag-and-drop functionality. Users can drag products and drop them into the cart area to add items. The interface provides an interactive and user-friendly shopping experience.

EX.NO:7	PRODUCT RATING IN APPLICATION
DATE:	

AIM:

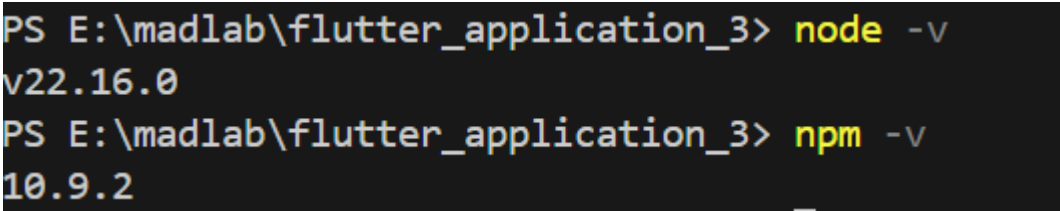
To develop an application that stores product details, number of reviews, and average ratings in Firebase Realtime Database, displays product information along with review ratings, and allows users to add reviews for each product.

STEPS TO CONNECT TO FIREBASE FROM VS CODE:

- 1. Check if node-js and npm is installed using the following commands:**

```
node -v
```

```
npm -v
```



```
PS E:\madlab\flutter_application_3> node -v
v22.16.0
PS E:\madlab\flutter_application_3> npm -v
10.9.2
```
- 2.Install firebase**

```
npm install -g firebase-tools
```

Check version

```
firebase --version
```
- 3.Install flutterfire**

```
dart pub global activate flutterfire_cli
```
- 4.Create a firebase project in Google account**

In firebase project, create realtime database and add data.
- 5.Login to firebase from vscode. Enter the following in vscode terminal.**

```
firebase login
```
- 6.Open flutter project folder in vscode terminal and execute the following command:**

```
flutterfire configure
```

Select the platforms and Enter

7.Add Firebase Core Package

```
flutter pub add firebase_core
```

PROGRAM:

```
// Firebase Reference
```

```
final DatabaseReference _dbRef = FirebaseDatabase.instance.ref("products");
```

```
// Read Data from Firebase
```

```
void readData() {
```

```
  _dbRef.onValue.listen((event) {
```

```
    final data = event.snapshot.value as Map?;
```

```
    if (data != null) {
```

```
      final temp = data.entries.map((e) => {
```

```
        "id": e.key,
```

```
        "name": e.value["name"],
```

```
        "price": e.value["price"],
```

```
        "rating": e.value["rating"] ?? 0,
```

```
        "count": e.value["users"] ?? 0,
```

```
        "reviews": e.value["reviews"] ?? {},
```

```
      }).toList();
```

```
      setState(() {
```

```
        products = temp;
```

```
    });
```

```
  }
```

```
});
```

```
}
```

```
// Update Rating (Average Calculation)
```

```
void updateRating(String productId, double newRating) async {
```

```
  DataSnapshot snapshot = await _dbRef.child(productId).get();
```

```

if (snapshot.exists) {
    Map data = snapshot.value as Map;
    double oldRating = double.tryParse(data["rating"].toString()) ?? 0.0;
    int oldCount = int.tryParse(data["users"].toString()) ?? 0;

    double updatedAverage =
        ((oldRating * oldCount) + newRating) / (oldCount + 1);
    await _dbRef.child(productId).update({
        "rating": double.parse(updatedAverage.toStringAsFixed(1)),
        "users": oldCount + 1,
    });
}
}

// Add Review to Firebase
void addReview(String productId) async {
    String reviewText = reviewControllers[productId]!.text.trim();
    if (reviewText.isEmpty)
        return;
    await _dbRef
        .child(productId)
        .child("reviews")
        .push()
        .set({"text": reviewText});

    reviewControllers[productId]!.clear();
}

// Display Rating Stars
Widget displayStars(double rating) {

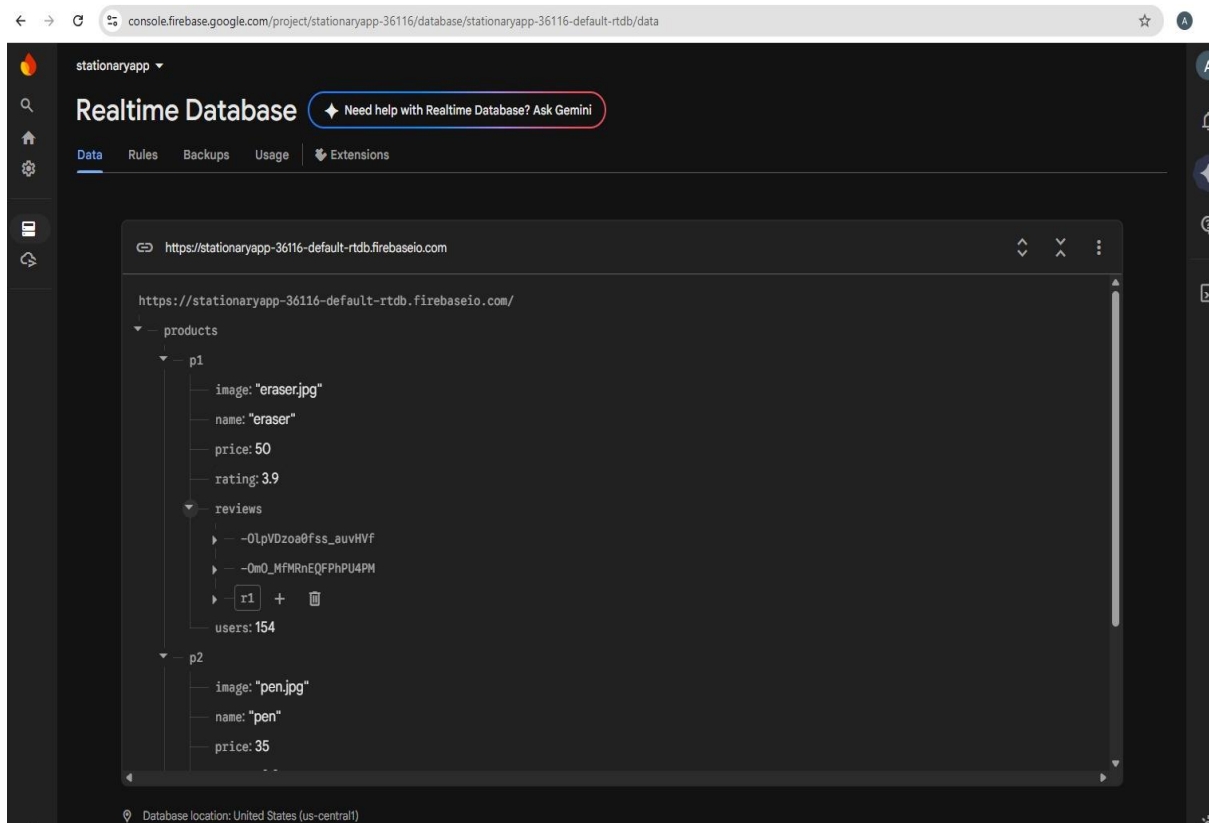
```

```

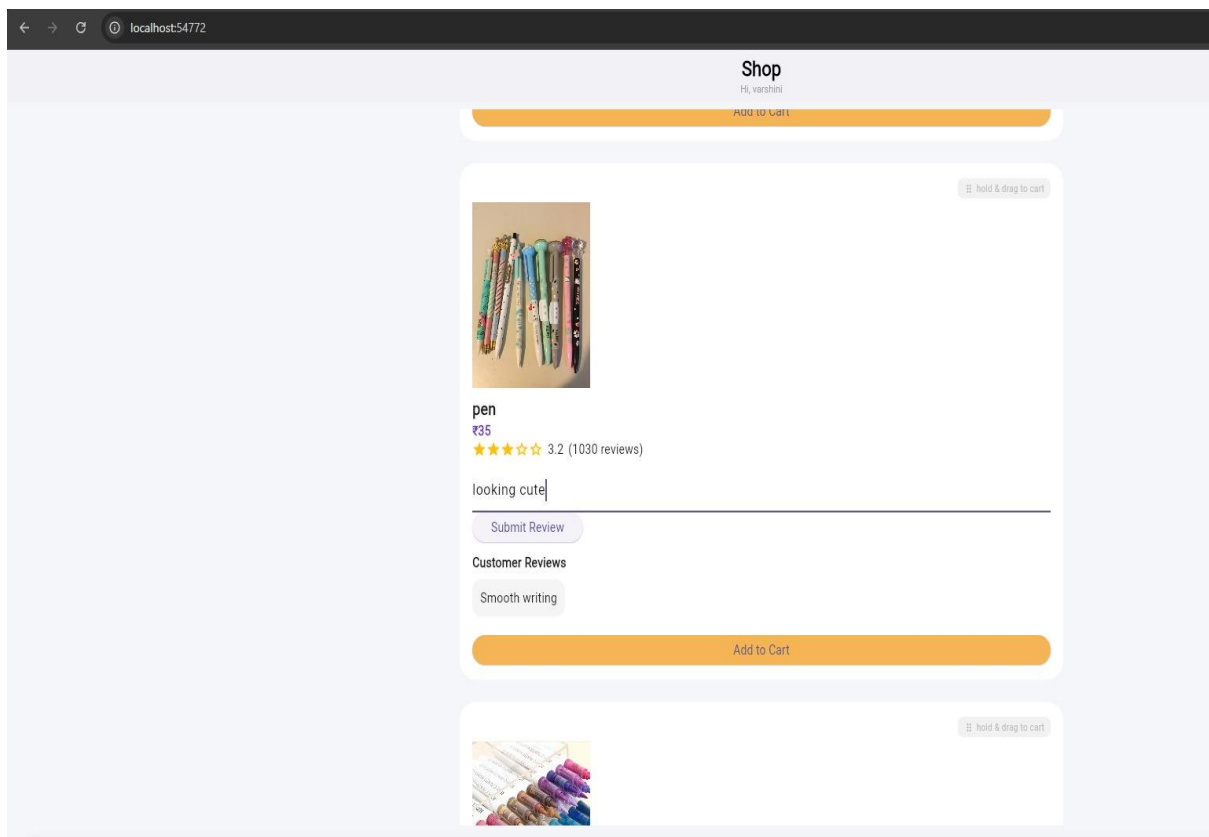
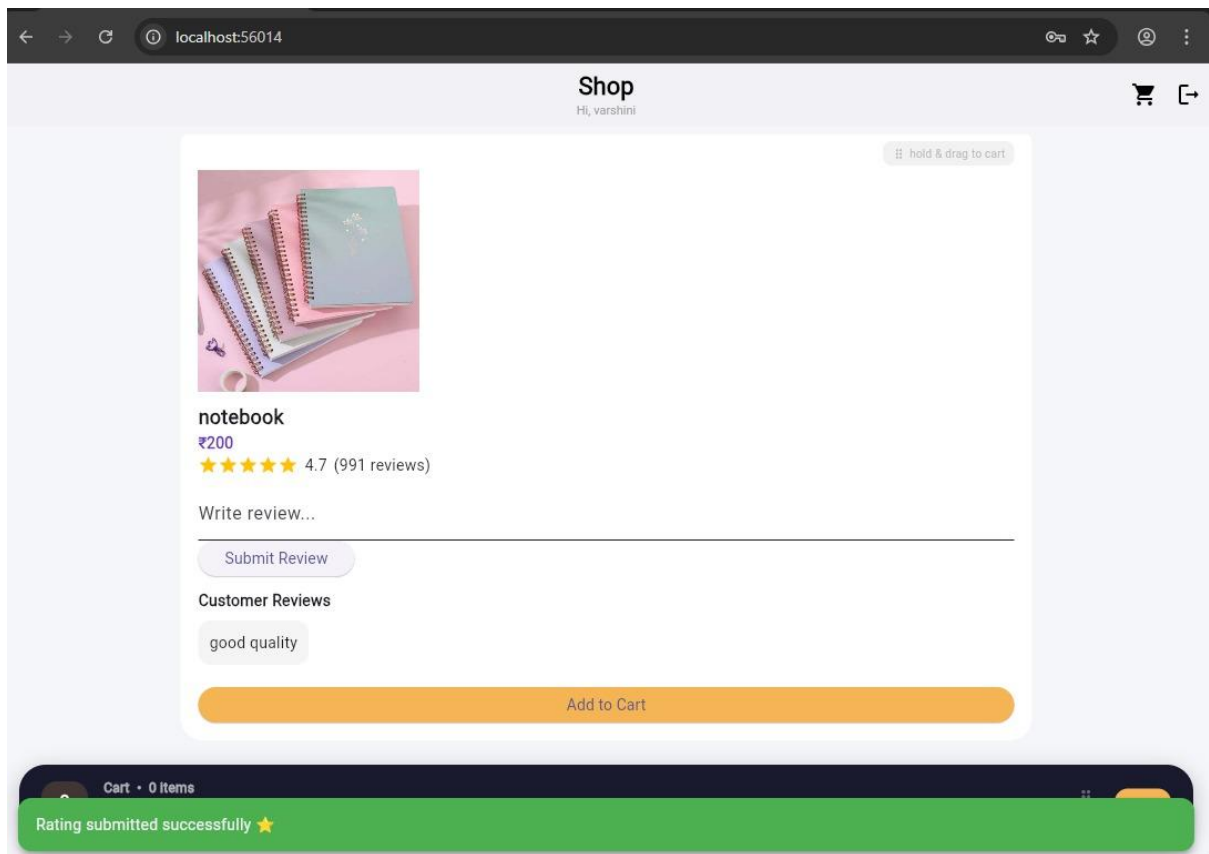
return Row(
  children: List.generate(
    5,
    (i) => Icon(
      i < rating.round() ? Icons.star : Icons.star_border,
      color: Colors.amber,
    ),
  ),
);
}

```

STORING DATA IN FIREBASE REALTIME DATABASE



OUTPUT:



RESULT:

Product details, reviews, and ratings were successfully stored in Firebase Realtime Database. The application displays product details along with average ratings. Users can add reviews, and data updates are reflected dynamically.

EX.NO:8	LOCATION SERVICE FOR MOBILE APPLICATION
DATE:	

AIM:

To develop a mobile application that utilizes location services to access and display the user's current location for providing location-based functionalities.

STEPS TO DISPLAY THE LOCATION(LATITUDE AND LONGITUDE)

Add dependency for geolocator in pubspec.yaml file

dependencies:

```
flutter:
  sdk: flutter
  geolocator: ^10.1.0
```

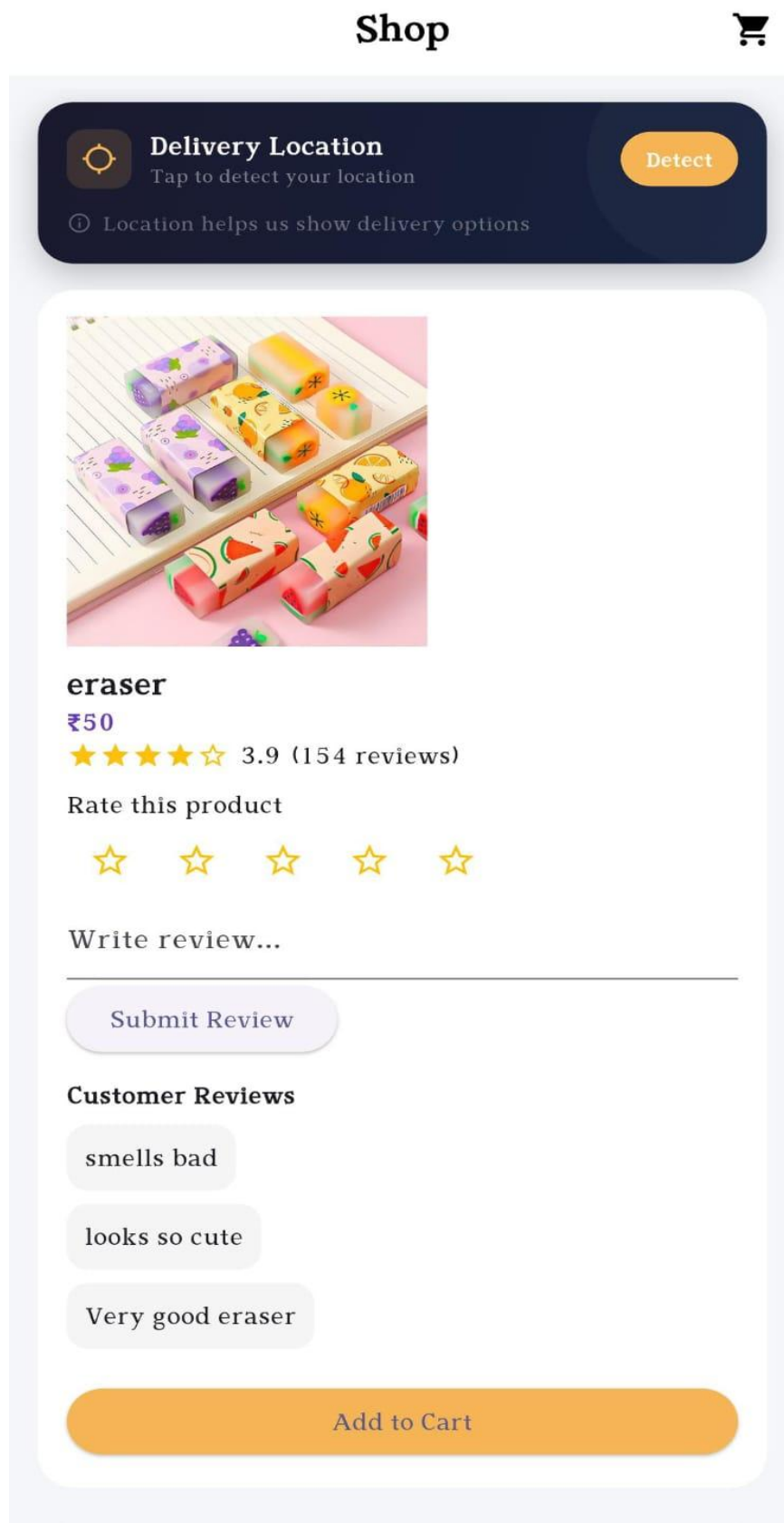
Check if location permission is enabled

```
permission = await Geolocator.checkPermission();
if (permission == LocationPermission.denied) {
  permission = await Geolocator.requestPermission();
}
```

Access the location

```
Position position =
  await Geolocator.getCurrentPosition();
setState() {
  location =
    "Latitude: ${position.latitude}\n"
    "Longitude: ${position.longitude}";
});
```

OUTPUT:



Shop



Delivery Location

GPS location detected

Refresh



5C87+MGC, Punnai Nagar, Tamil Nadu, 629004, India

LAT 8.16722

LNG 77.41407

±100m



eraser

₹50

★★★★☆ 3.9 (154 reviews)

Rate this product



Write review...

Submit Review

Customer Reviews

smells bad

looks so cute

Very good eraser

Add to Cart

RESULT:

The mobile application successfully accessed and displayed the user's current location. Location services were integrated to provide location-based functionality. The application responded accurately using real-time location data.

EX.NO:9	DESIGN A WEB SERVER SUPPORTING PUSH NOTIFICATIONS
DATE:	

AIM:

To design and implement a web server that supports push notifications, enabling real-time communication by sending updates and alerts to users’ devices.

STEPS TO PUSH NOTIFICATION:

1.Add firebase messaging to flutter project using the following commands:

dart pub global activate flutterfire_cli

flutterfire configure

flutter pub add firebase_messaging

2.Configure firebase web

Change web->index.html

<script src="https://www.gstatic.com/firebasejs/10.7.0/firebase-app-compat.js"></script>

<script src="https://www.gstatic.com/firebasejs/10.7.0/firebase-messaging-compat.js"></script>

3.Add the js file named “firebase-messaging-sw.js”

API key and project details to be copied from firebase project settings

importScripts("https://www.gstatic.com/firebasejs/10.7.0/firebase-app-compat.js");

importScripts("https://www.gstatic.com/firebasejs/10.7.0/firebase-messaging-compat.js");

firebase.initializeApp({

apiKey: "AIzaSyD6a4P5HMLpi1qNoyMCIDBtY2SjZFmgdA",

authDomain: "stationaryapp-36116.firebaseio.com",

databaseURL: "https://stationaryapp-36116-default-rtdb.firebaseio.com",

projectId: "stationaryapp-36116",

storageBucket: "stationaryapp-36116.firebaseio.com",

messagingSenderId: "601250993481",

```
appId: "1:601250993481:web:6dcb66a8f4fdb608423698",  
measurementId: "G-52X6GRYZCH"  
});
```

4.Add the following code in main.dart

Change the vapid key

Vapid key should be generated from firebase messaging settings

Create web push certificate and copy public key

main.dart:

```
import 'package:flutter/material.dart';  
import 'package:firebase_core/firebase_core.dart';  
import 'package:firebase_messaging/firebase_messaging.dart';  
import 'firebase_options.dart';  
  
// Background handler  
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {  
  print("Background message: ${message.notification?.title}");  
}  
  
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);  
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);  
  runApp(MyApp());  
}  
  
class MyApp extends StatefulWidget {  
  @override  
  State<MyApp> createState() => _MyAppState();  
}  
  
class _MyAppState extends State<MyApp> {  
  String token = "Fetching token...";
```

```

@Override
void initState() {
    super.initState();
    setupFCM();
}

void setupFCM() async {
    FirebaseMessaging messaging = FirebaseMessaging.instance;

    // Request permission (important for Web)
    await messaging.requestPermission();

    // Use VAPID KEY here
    String? t = await messaging.getToken(
        vapidKey:
            "BCy0bv9hWo1H1fo8uhXTPNWruNsHfWpQLg3s1x_ckwfEGl5rYYZUf4V_csj3GEq
1MBc9xXiOkQAmyYER09Wl748",
    );

    setState() {
        token = t ?? "No token";
    });

    print("FCM Token: $t");

    // Foreground messages
    FirebaseMessaging.onMessage.listen((RemoteMessage message) {
        print("Foreground message: ${message.notification?.title}");
    });

    // Click notification
    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
        print("Notification clicked!");
    });
}

```

@override

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(title: Text("Flutter Web FCM")),  
      body: Padding(  
        padding: EdgeInsets.all(16),  
        child: SelectableText("Token:\n\n$token"),  
      ),  
    ),  
  );  
}
```

5. Open firebase -> cloud messaging -> new campaign

Notification title ⓘ

notification

Notification text

Fresh arrivals! Our new journaling collection is here. Come see what's new

Notification image (optional) ⓘ

Example: <https://yourapp.com/image.png>

Notification name (optional) ⓘ

Enter optional name

Next

Device preview

This preview provides a general idea of how your message will appear on a mobile device. Actual message rendering will vary depending on the device. Test with a real device for actual results.

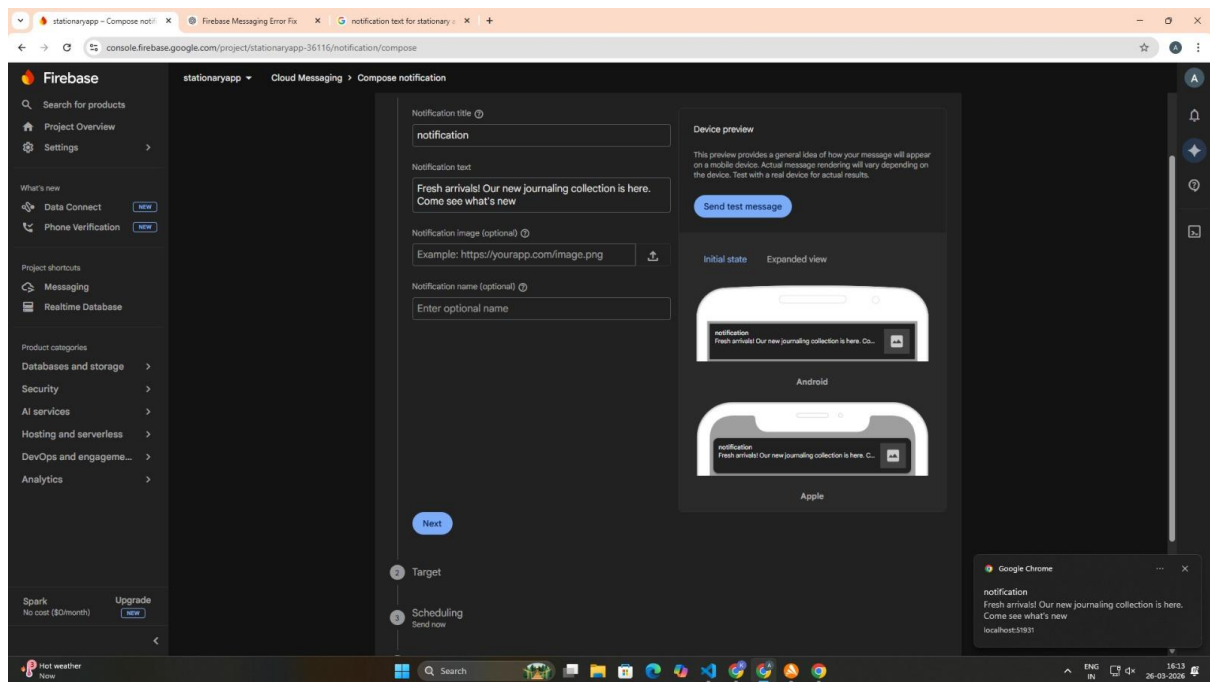
Send test message

Initial state Expanded view

Android

Apple

OUTPUT:



RESULT:

The web server was successfully designed to support push notifications. It sends real-time updates and alerts to connected users. The system ensures efficient and timely communication between server and clients.